

Конспект по авторизации

Проект **multi-tenant-ecommerce**: бэкенд на NestJS + Prisma, фронтенд **platform-admin** на React + Redux

Пошаговый разбор: как у вас устроена авторизация, что и где реализовано на бэкенде и фронтенде, какие понятия нужно знать, что запомнить и на что обратить внимание.

Репозиторий	github.com/ShipaM/multi-tenant-ecommerce
Состояние кода	коммит ee4653a от 2026-09-29 (ветка по умолчанию)
Что разобрано	backend/src/auth , backend/src/users , backend/src/common , backend/prisma , platform-admin/src (store, lib, api, routes, pages)
Для кого	для вас, автора проекта: стартовый учебный проект, цель — понять, как всё работает, и уметь это объяснить

ЗАПОМНИ Как читать конспект

Части 1–4 дают общую картину и словарь. Части 5–6 — устройство бэкенда и фронтенда по слоям. Часть 7 — главная: пошаговые сценарии (вход, 2FA, refresh, выход, смена и сброс пароля). Части 8–9 — что сделано хорошо и что стоит улучшить. Часть 10 — чек-лист и вопросы для самопроверки.

Рамки: **ЗАПОМНИ** — главное, **ВНИМАНИЕ** — подводные камни, **ИДЕЯ** — как улучшить. Пути к файлам указаны от корня репозитория.

Содержание

1. Картина целиком	4
1.1. Из чего состоит проект	4
1.2. Общая схема	4
1.3. Что умеет авторизация в проекте	4
2. Ключевые понятия	5
3. База данных (Prisma)	7
3.1. Таблицы и поля, которые важно понимать	8
3.2. История миграций	8
4. Четыре вида JWT	9
4.1. Зачем два токена (access + refresh)	9
4.2. Временная шкала входа	9
5. Бэкенд по слоям (NestJS)	10
5.1. Путь запроса через сервер	10
5.2. main.ts: глобальные настройки	10
5.3. app.module.ts, конфигурация и .env	10
5.4. Эндпоинты авторизации	11
5.5. AuthService: что делает каждый метод	12
5.6. Как работает защита маршрута: Guard + Strategy + Decorator	13
5.7. UsersService	13
5.8. DTO и проверка входных данных	14
5.9. Прочее: User-Agent, почта, фильтр ошибок	14
6. Фронтенд по слоям (platform-admin)	15
6.1. Карта файлов фронтенда	15
6.2. Хранение токенов: lib/storage.ts	15
6.3. lib/axios.ts: сердце фронтенд-авторизации	16
6.4. Redux: authSlice	16
6.5. Маршруты и защита страниц	17
6.6. Экраны авторизации и как они общаются	17
7. Пошаговые сценарии	18
7.1. Вход без 2FA	18
7.2. Вход с двухфакторной аутентификацией	19
7.3. Запрос к защищённому эндпоинту	20
7.4. Автоматическое обновление токена (refresh)	20
7.5. Выход и управление сессиями	21
7.6. Включение и выключение 2FA	21
7.7. Смена пароля (вы знаете текущий)	22
7.8. Восстановление пароля (забыли пароль)	22

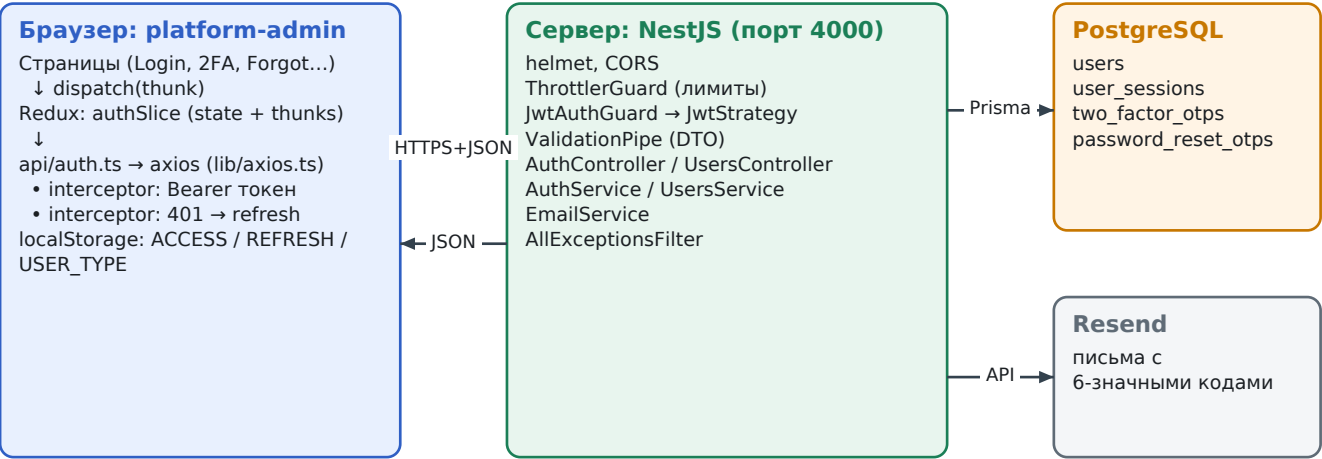
- 7.9. Редактирование профиля 23
- 8. Что в вашей реализации сделано хорошо 23**
- 9. На что обратить внимание: замечания к текущему коду 24**
- 9.1. Что нужно для настоящей мультитенантности 26
- 10. Что запомнить и как проверить себя 26**
- 10.1. Чек-лист: 15 главных мыслей 26
- 10.2. Вопросы для самопроверки 26
- Приложения 28**
- A. Как увидеть всё это своими глазами 28
- B. Карта файлов авторизации 29
- C. Если нужно больше деталей 29

1. Картина целиком

1.1. Из чего состоит проект

Часть	Технологии	Роль в авторизации
backend/ (порт 4000)	NestJS 12, Prisma 7 + PostgreSQL, passport-jwt, @nestjs/jwt, bcryptjs, @nestjs/throttler, helmet, class-validator, Resend (почта), ua-parser-js, vitest	Проверяет пароль, выдаёт и проверяет токены, хранит сессии и одноразовые коды, отправляет письма с кодами
platform-admin/ (порт 3002)	React 19, Vite, react-router, Redux Toolkit, axios, shadcn/ui, sonner, moment	Формы входа, хранение токенов, автоматическое обновление токена, защита страниц, страница «Мой аккаунт»
PostgreSQL	таблицы <code>users</code> , <code>user_sessions</code> , <code>two_factor_otps</code> , <code>password_reset_otps</code>	Единственный источник правды о пользователях и сессиях
Resend	внешний почтовый сервис	Доставляет 6-значные коды (2FA и сброс пароля)

1.2. Общая схема



1.3. Что умеет авторизация в проекте

- **Вход по email + пароль** (`POST /auth/login`) с выдачей пары токенов и созданием сессии.
- **Двухфакторная аутентификация (2FA)** по коду из письма: и при входе, и как включаемая/выключаемая настройка.
- **Access + refresh токены** с автоматическим обновлением на фронтенде и **ротацией** refresh-токена.
- **Серверные сессии** (таблица `user_sessions`): список устройств, выход с конкретного устройства, выход со всех остальных.
- **Выход** (`POST /auth/logout`) с мгновенной инвалидацией токена.
- **Смена пароля** (знаете текущий) и **восстановление пароля** (забыли) через код из письма.
- **Профиль** (`PUT /users/me`): имя, email, телефон, фото.
- **Защита от перебора**: лимиты запросов, лимит попыток ввода кода, одинаковые ответы для несуществующих email.

ВНИМАНИЕ Про «multi-tenant»: уточнение

В названии проекта есть multi-tenant, но в **коде авторизации понятия «тенант» (магазин/организация) пока нет**: в схеме БД нет таблицы тенантов, в токенах нет `tenantId`. Есть только поле `userId` (`CUSTOMER`, `SELLER`, `PLATFORM_ADMIN`, `DELIVERY_AGENT`).

Фронтенд `platform-admin` — консоль владельца платформы (текст на экране входа: «Run every store on your marketplace from one place»). Изоляция данных между тенантами — это следующий этап; ниже, в части 9, я отмечу, что для него нужно добавить.

2. Ключевые понятия

Это словарь, к которому удобно возвращаться. В колонке «Где в проекте» указано, где вы это используете.

Основы

Понятие	Что это	Где в проекте
Аутентификация (authN)	Выяснить, кто перед нами: проверить email и пароль, код из письма.	<code>AuthService.login</code> , <code>JwtStrategy</code>
Авторизация (authZ)	Решить, что этому пользователю можно : роли, права, доступ к данным тенанта.	В проекте пока не реализована (см. часть 9)
Сессия	Запись «этот пользователь вошёл на этом устройстве». Позволяет отозвать вход.	таблица <code>user_sessions</code>
Stateless / stateful	Stateless — сервер ничего не хранит и верит подписи токена. Stateful — сервер хранит состояние (сессию) и сверяется с ним.	Гибрид: JWT + проверка сессии в БД на каждый запрос

Токены

Понятие	Что это	Где в проекте
JWT	Строка <code>header.payload.signature</code> . Payload — обычный base64url (его может прочитать любой), подпись не даёт его подделать.	<code>JwtService.signAsync</code> / <code>verifyAsync</code>
Access token	Короткоживущий (15 мин) токен для запросов к API в заголовке <code>Authorization: Bearer</code>	<code>signTokenPair</code> , <code>JwtStrategy</code>
Refresh token	Долгоживущий (30 дней) токен, нужен только чтобы получить новую пару токенов.	<code>AuthService.refresh</code> , <code>lib/axios.ts</code>
Ротация refresh	При каждом обновлении выдаётся новый refresh, а старый перестаёт работать.	<code>refresh()</code> → <code>userSession.update</code>
Bearer	Схема передачи токена: <code>Authorization: Bearer <токен></code> .	interceptor в <code>lib/axios.ts</code>
Секрет подписи	Строка ≥ 32 символов, которой подписывают JWT. У каждого вида токена свой.	<code>JWT_*_SECRET</code> в <code>.env</code>

Пароли, коды и хеши

Понятие	Что это	Где в проекте
Хеш	Необратимое «отпечаток» данных. Пароль в БД хранится только в виде хеша.	<code>passwordHash</code>
bcrypt	Медленный хеш для паролей с «солью»; параметр <code>cost=10</code> задаёт, насколько он медленный.	<code>bcrypt.hash(pw, 10)</code>
HMAC-SHA256	Хеш с секретным ключом. Им хешируется refresh-токен перед записью в БД.	<code>hashRefreshToken</code>
OTP	One-Time Password: одноразовый 6-значный код, живёт 5 минут, допускает 5 попыток.	<code>two_factor_otps</code> , <code>password_reset_otps</code>
2FA	Второй фактор: кроме знания пароля нужно подтвердить владение почтой.	<code>twoFactorEnabled</code>
Timing attack	Угадывание по времени ответа. Защита: сравнение за постоянное время.	<code>timingSafeEqual</code> , dummy-хеш
Email enumeration	Выяснение, зарегистрирован ли email, по разнице в ответах.	одинаковые ответы в <code>login</code> , <code>forgotPassword</code>

Бэкенд (NestJS)

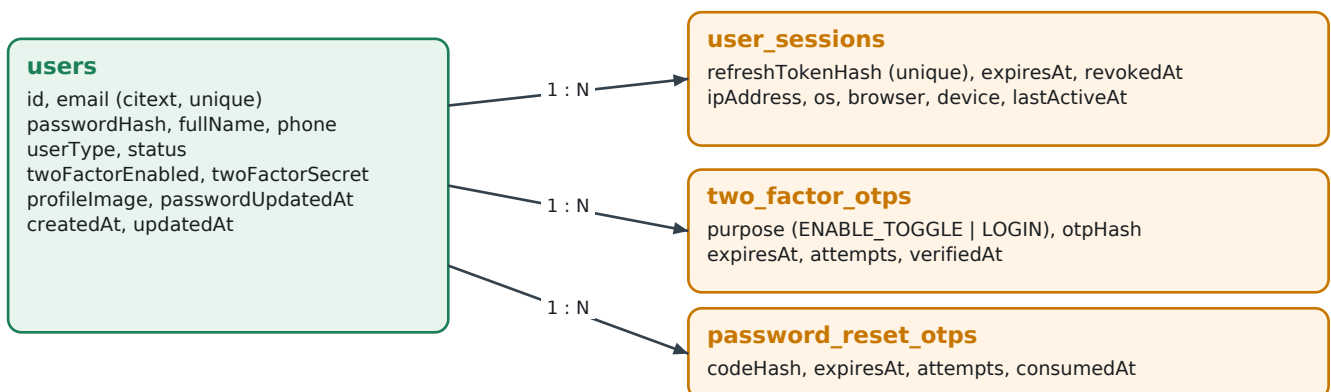
Понятие	Что это	Где в проекте
Module / Controller / Service	Модуль группирует код; контроллер принимает HTTP-запрос; сервис содержит логику.	<code>AuthModule</code> , <code>AuthController</code> , <code>AuthService</code>
DTO + ValidationPipe	DTO — класс-описание тела запроса; pipe проверяет и чистит данные до контроллера.	<code>LoginDto</code> , <code>main.ts</code>
Guard	Решает, пустить ли запрос в обработчик. Выполняется до pipes.	<code>JwtAuthGuard</code> , <code>ThrottlerGuard</code>
Passport Strategy	Правило проверки учётных данных запроса (здесь — JWT из заголовка).	<code>JwtStrategy</code>
Decorator	<code>@Post</code> , <code>@UseGuards</code> , <code>@Throttle</code> , <code>@CurrentUser</code> — метки/инструменты над методами и параметрами.	<code>current-user.decorator.ts</code>
Exception Filter	Единое место, где превращаются ошибки в HTTP-ответы.	<code>AllExceptionsFilter</code>
Prisma	ORM: описываете схему в <code>schema.prisma</code> , получаете типизированный клиент.	<code>PrismaService</code>

Фронтенд (React)

Понятие	Что это	Где в проекте
Redux Toolkit slice	Хранилище состояния + редьюсеры для одной области (auth).	<code>store/auth/authSlice.ts</code>
<code>createAsyncThunk</code>	Асинхронное действие с состояниями <code>pending</code> / <code>fulfilled</code> / <code>rejected</code> .	<code>fetchLogin</code> , <code>fetchMe...</code>
<code>unwrap()</code>	Превращает результат <code>thunk</code> в обычный промис: успех → значение, ошибка → <code>throw</code> .	<code>dispatch(...).unwrap()</code> на страницах
Interceptor (axios)	Перехватчик каждого запроса/ответа: добавляет токен, ловит 401.	<code>lib/axios.ts</code>
Protected route	Маршрут-обёртка, который не пускает без токена.	<code>ProtectedRoute.tsx</code>
<code>localStorage</code>	Хранилище браузера, переживает перезагрузку, доступно любому JS на странице.	<code>lib/storage.ts</code>
Open redirect	Атака через параметр вида <code>?redirect_uri=https://evil.com</code> .	<code>isSafeRedirectPath</code>

3. База данных (Prisma)

Файл: `backend/prisma/schema.prisma`. Все связанные таблицы ссылаются на `users` и имеют `onDelete: Cascade` — при удалении пользователя его сессии и коды удаляются автоматически.



3.1. Таблицы и поля, которые важно понимать

Таблица.поле	Значение
<code>users.email</code>	Тип <code>citext</code> (расширение PostgreSQL) — сравнение без учёта регистра , плюс <code>@unique</code> . <code>Admin@x.com</code> и <code>admin@x.com</code> — один и тот же пользователь.
<code>users.passwordHash</code>	bcrypt-хеш пароля. Никогда не отдаётся клиенту (в ответах используется <code>omit</code>).
<code>users.userType</code>	Тип пользователя (enum). Сейчас только хранится и кладётся в токен , но доступ по нему не проверяется.
<code>users.status</code>	<code>ACTIVE</code> / <code>SUSPENDED</code> / <code>INACTIVE</code> . Войти и пользоваться API может только <code>ACTIVE</code> (проверяется при входе, при refresh и на каждом запросе).
<code>users.twoFactorEnabled</code>	Включена ли 2FA. Если да — после пароля требуется код из письма.
<code>users.twoFactorSecret</code>	Зарезервировано под TOTP-приложения (Google Authenticator). Сейчас не используется : коды идут по почте.
<code>users.passwordUpdatedAt</code>	Когда последний раз менялся пароль (показывается на странице аккаунта: «changed ... ago»).
<code>user_sessions</code> (строка)	Один вход на одном устройстве. <code>id</code> попадает в access-токен как <code>sid</code> .
<code>user_sessions.refreshTokenHash</code>	HMAC-хеш текущего refresh-токена (сам токен не хранится).
<code>user_sessions.expiresAt</code>	Срок жизни сессии = <code>exp</code> refresh-токена. Обновляется при каждом refresh.
<code>user_sessions.revokedAt</code>	Если не <code>null</code> — сессия отозвана, токены перестают работать немедленно.
<code>user_sessions.os / browser / device / ipAddress</code>	Разбор <code>User-Agent</code> (ua-parser-js) и IP — для списка устройств.
<code>two_factor_otps.purpose</code>	<code>LOGIN</code> (код для входа) или <code>ENABLE_TOGGLE</code> (код для включения/выключения 2FA). Код одной цели не подходит для другой.
<code>*_otps.otpHash / codeHash</code>	bcrypt-хеш 6-значного кода: в БД нет самого кода.
<code>*_otps.attempts</code>	Счётчик неверных попыток. После 5 — «Too many otp attempts».
<code>two_factor_otps.verifiedAt, password_reset_otps.consumedAt</code>	Когда код был успешно использован: делает код одноразовым.

3.2. История миграций

Миграция	Что добавила
20260907 <code>users_session_tables</code>	<code>citext</code> , enum-ы, <code>users</code> , <code>user_sessions</code> , <code>password_reset_otps</code>
20260910 <code>profile_image</code>	<code>users.profile_image</code>
20260917 <code>two_factor_otp_and_expiry_indexes</code>	<code>two_factor_otps</code> , индексы по <code>expires_at</code>
20260918 <code>two_factor_otp_purpose</code>	поле <code>purpose</code> (<code>LOGIN</code> / <code>ENABLE_TOGGLE</code>)
20260922 <code>user_agent</code>	<code>os</code> , <code>device</code> , <code>browser</code> в сессии
20260924 <code>added_password_updated_at_field</code>	<code>users.password_updated_at</code>
20260929 <code>password_reset_otp_attempts</code>	<code>attempts</code> в <code>password_reset_otps</code>

ЗАПОМНИ Главная идея схемы

Пользователь ≠ сессия ≠ токен. **Токен — это «пропуск», сессия — «запись в журнале выдачи пропусков».** Пропуск валиден только пока запись в журнале не отозвана и не истекла — поэтому выход и блокировка работают мгновенно, а не через 15 минут.

4. Четыре вида JWT

В проекте подписываются **четыре разных токена**, у каждого свой секрет и срок жизни. Разные секреты нужны, чтобы токен одного вида нельзя было выдать за токен другого.

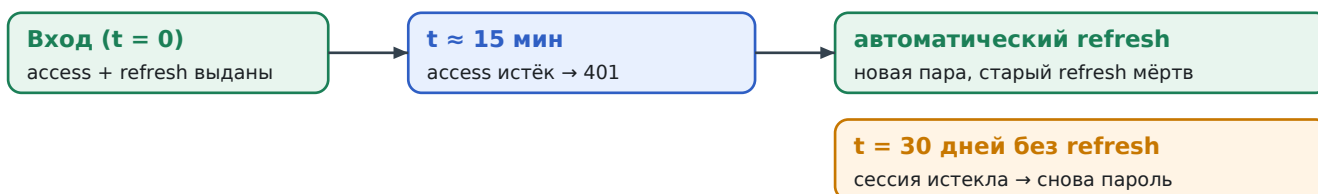
Токен	Секрет (.env)	TTL	Payload	Где хранится	Зачем
Access	JWT_ACCESS_SECRET	15m	userId, email, userType, sid	Клиент: localStorage. Сервер: нигде	Доступ к защищённым эндпоинтам (Bearer)
Refresh	JWT_REFRESH_SECRET	30d	userId, sessionId	Клиент: localStorage. Сервер: только HMAC-хеш в user_sessions	Получить новую пару токенов
2FA	JWT_2FA_SECRET	15m	userId	Клиент: location.state роутера. Сервер: нигде	twoFactorToken: доказать «пароль уже проверен», пока ждём код из письма
Reset	JWT_RESET_SECRET	5m	userId	Клиент: location.state. Сервер: нигде	resetToken: право задать новый пароль после ввода кода

TTL — срок жизни по умолчанию. Значения можно менять в .env (формат 15m, 7d, 3600s, проверяется регулярным выражением в env.validation.ts).

4.1. Зачем два токена (access + refresh)

- Access-токен уходит в **каждом** запросе — вероятность его утечки выше. Поэтому он живёт 15 минут: украденный токен быстро становится бесполезным.
- Refresh-токен используется **редко** (раз в ~15 минут и только на /auth/refresh). Он живёт долго, чтобы пользователь не вводил пароль каждые 15 минут.
- На сервере refresh привязан к сессии: его можно отозвать (revokedAt), а при каждом использовании он заменяется.

4.2. Временная шкала входа



ЗАПОМНИ JWT подписан, но не зашифрован

Любой может декодировать payload токена (это просто base64url). Подпись защищает от **подделки**, а не от **чтения**. Поэтому в токен нельзя класть пароли и секреты. В вашем access-токене лежат **userId**, **email**, **userType** и **sid** — это нормально.

Практика: откройте DevTools → Application → Local Storage, скопируйте **ACCESS_TOKEN** и выполните в консоли `JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))` — увидите payload и поле **exp**.

5. Бэкенд по слоям (NestJS)

5.1. Путь запроса через сервер

Порядок важен: если запрос «отваливается» на раннем шаге, до следующих он не доходит.

1. **helmet** (**main.ts**) — добавляет защитные HTTP-заголовки ко всем ответам.
2. **CORS** (**main.ts**) — разрешает запросы только с origin из **CORS_ORIGINS** (например `http://localhost:3002`). Если переменная не задана, CORS выключен.
3. **ThrottlerGuard** (глобально, **app.module.ts**) — лимит запросов: по умолчанию 60 в минуту; на чувствительных маршрутах свои, более строгие лимиты (**@Throttle**).
4. **JwtAuthGuard** (только на защищённых маршрутах через **@UseGuards**) — проверка токена и сессии.
5. **ValidationPipe** (глобально) — проверка тела запроса по DTO.
6. **Controller** → **Service** → **Prisma** — бизнес-логика и база.
7. **AllExceptionsFilter** — любую ошибку превращает в единый JSON { **statusCode**, **message**, **error**, **path**, **timestamp** }; необработанные ошибки наружу как «Internal server error» (без стека), а стек пишется в лог.

5.2. **main.ts**: глобальные настройки

```
app.use(helmet());
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,           // выбросить поля, которых нет в DTO
  forbidNonWhitelisted: true, // а если лишние поля есть — вернуть 400
  transform: true,           // превратить JSON в экземпляр DTO-класса
  transformOptions: { enableImplicitConversion: true },
}));
app.useGlobalFilters(new AllExceptionsFilter());
if (corsOrigins) app.enableCors({ origin: [...], credentials: true });
```

ЗАПОМНИ whitelist + forbidNonWhitelisted = защита от mass assignment

В **UserService.update** в Prisma уходит **data: { ...dto }**. Это безопасно только потому, что pipe не пропустит в DTO лишние поля. Иначе клиент мог бы отправить { **"userType": "PLATFORM_ADMIN"** } и повысить себе права.

5.3. **app.module.ts**, конфигурация и **.env**

- **ConfigModule.forRoot**({ **isGlobal**, **cache**, **validate: validateEnv** }) — переменные окружения проверяются **при старте**: нет секрета или он короче 32 символов → приложение не запустится (**env.validation.ts**).
- **ThrottlerModule.forRoot** + **APP_GUARD: ThrottlerGuard** — глобальный лимит 60 запросов/мин.
- **GlobalPassportModule** — регистрирует Passport со стратегией по умолчанию **jwt**.

Переменная	Нужна?	Значение
DATABASE_URL	да	строка подключения PostgreSQL

Переменная	Нужна?	Значение
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, JWT_2FA_SECRET, JWT_RESET_SECRET	да	четыре разных случайных строки, каждая ≥ 32 символов
JWT_ACCESS_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN / JWT_2FA_EXPIRES_IN / JWT_RESET_EXPIRES_IN	нет	по умолчанию 15m / 30d / 15m / 5m
RESEND_API_KEY, RESEND_FROM_EMAIL	да	ключ и адрес отправителя для писем
CORS_ORIGINS	нет	список origin через запятую; без неё CORS выключен
PORT, NODE_ENV	нет	по умолчанию 4000 и development

5.4. Эндпоинты авторизации

Метод и путь	Защита	Лимит	Тело	Что делает
POST /auth/login	публичный	10	email, password	Проверяет пароль. Возвращает токены или требование 2FA
POST /auth/refresh	публичный	20	refreshToken	Ротация: новая пара токенов
POST /auth/logout	Bearer	60	—	Отзывает текущую сессию
GET /auth/me	Bearer	60	—	Профиль без passwordHash и twoFactorSecret
POST /auth/2fa-generate-otp	Bearer	5	—	Шлёт код для включения/выключения 2FA
POST /auth/2fa-verify-enable	Bearer	5	otp	Проверяет код и переключает twoFactorEnabled
POST /auth/2fa-verify-login-otp	публичный	5	otp, twoFactorToken	Завершает вход с 2FA (личность подтверждает twoFactorToken), выдаёт токены
GET /auth/sessions	Bearer	60	—	Активные сессии пользователя
POST /auth/sessions/:id/revoke	Bearer	60	—	Отозвать одну сессию
POST /auth/sessions/revoke-others	Bearer	60	—	Отозвать все, кроме текущей
POST /auth/forgot-password	публичный	5	email	Шлёт код сброса на почту
POST /auth/forgot-password/verify-otp	публичный	5	email, otp	Проверяет код → resetToken
POST /auth/reset-password	публичный	5	resetToken, password	Задаёт новый пароль (право даёт resetToken)
PUT /users/me	Bearer	60	fullName, email, phone, profileImage	Обновить профиль
PUT /users/change-password	Bearer	60	currentPassword, password	Сменить пароль

«Лимит» — запросов в минуту; 60 — глобальное значение по умолчанию. Throttler по умолчанию считает запросы по IP.

5.5. AuthService: что делает каждый метод

Метод	Суть
<code>login(email, password, ctx)</code>	Ищет пользователя; всегда выполняет <code>bcrypt.compare</code> (для несуществующего пользователя — с заранее заготовленным dummy-хешем); неверно → 401 «Invalid email or password»; статус не ACTIVE → 403; при включённой 2FA возвращает <code>twoFactorRequired</code> вместо токенов; иначе <code>issueTokenPair</code> .
<code>issueTokenPair(user, ctx)</code>	Создаёт <code>sessionId</code> (<code>cuid2</code>), подписывает пару токенов и вставляет строку в <code>user_sessions</code> (хеш <code>refresh</code> , срок из <code>exp</code> , устройство, IP).
<code>signTokenPair(user, sessionId)</code>	Параллельно (<code>Promise.all</code>) подписывает <code>access</code> и <code>refresh</code> — каждый своим секретом и сроком.
<code>refresh(token, ctx)</code>	4 проверки: (1) подпись и срок JWT; (2) сессия существует, не отозвана, не истекла; (3) хеш токена совпадает с хешем в БД (<code>timingSafeEqual</code>); (4) пользователь ACTIVE . Затем выдаёт новую пару и перезаписывает хеш и срок в той же сессии.
<code>logout(userId, sessionId)</code>	Проверяет, что сессия принадлежит пользователю; ставит <code>revokedAt</code> .
<code>me(userId)</code>	Возвращает пользователя с <code>omit: { passwordHash, twoFactorSecret }</code> .
<code>twoFactorEnable(userId, purpose)</code>	Генерирует код <code>randomInt(100000, 1000000)</code> , хеширует <code>bcrypt</code> , гасит прежние живые коды той же цели, создаёт запись (TTL 5 мин), отправляет письмо.
<code>verifyOtp(userId, purpose, otp) (private)</code>	Общая проверка кода для обоих сценариев 2FA: берёт новейший неиспользованный и неистёкший код нужной цели; ≥ 5 попыток → 400; неверный → <code>attempts + 1</code> ; верный → <code>verifiedAt = now</code> .
<code>twoFactorVerifyEnable</code>	Проверка кода <code>ENABLE_TOGGLE</code> и инверсия <code>twoFactorEnabled</code> .
<code>twoFactorCreateOtpToken</code>	Шлёт код LOGIN и подписывает <code>twoFactorToken</code> .
<code>twoFactorVerifyLoginOtp</code>	Проверяет <code>twoFactorToken</code> , статус пользователя и код LOGIN , затем <code>issueTokenPair</code> .
<code>listSessions / revokeSession / revokeOtherSessions</code>	Работа со списком устройств (только свои сессии).
<code>forgotPassword / forgotPasswordOtpVerification / resetPassword</code>	Трёхшаговое восстановление пароля (часть 7.8).

5.6. Как работает защита маршрута: Guard + Strategy + Decorator

```
// common/guards/jwt-auth.guard.ts
export class JwtAuthGuard extends AuthGuard('jwt') {}

// auth/strategies/jwt.strategy.ts
super({ jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  ignoreExpiration: false,
  secretOrKey: config.getOrThrow('JWT_ACCESS_SECRET') });

async validate(payload: JwtAccessPayload): Promise<AuthenticatedUser> {
  const session = await this.prisma.userSession.findUnique({
    where: { id: payload.sid }, include: { user: true } });
  if (!session || session.revokedAt || session.expiresAt <= new Date())
    throw new UnauthorizedException('Session is no longer valid');
  if (session.user.status !== UserStatus.ACTIVE)
    throw new UnauthorizedException('This account is not active');
  return { userId: session.userId, email: session.user.email,
    userType: session.user.userType, sessionId: session.id };
}

// в контроллере
@UseGuards(JwtAuthGuard)
@Get('me') me(@CurrentUser() user: AuthenticatedUser) {...}
```

- `@UseGuards(JwtAuthGuard)` → Passport вызывает стратегию `jwt`. Она достаёт токен из заголовка и проверяет **подпись и срок**.
- Если токен валиден, вызывается `validate(payload)`. Вы дополнительно идёте в **БД** за сессией. Это и есть мост между «stateless JWT» и «управляемой сессией».
- Возвращённый из `validate` объект Nest кладёт в `request.user`; декоратор `@CurrentUser()` достаёт его в параметр метода.
- `email` и `userType` берутся **из БД**, а не из токена — значит их изменение вступает в силу сразу.

ЗАПОМНИ Цена мгновенного logout

Каждый защищённый запрос делает запрос в БД за сессией. Это осознанный компромисс: чистый JWT не требует БД, но не позволяет «выкинуть» пользователя до истечения токена. Ваш выбор — безопасность и управляемость.

5.7. UsersService

Метод	Суть
<code>findByEmail</code>	<code>findUnique</code> по email (citext → без учёта регистра).
<code>update(userId, dto)</code>	Обновляет профиль, не возвращая секретные поля. Нарушение уникальности email (Prisma P2002) → 409 «Email is already in use».
<code>changePassword</code>	Сверяет <code>currentPassword</code> (иначе 401), хеширует новый (bcrypt, cost 10) и в одной транзакции обновляет пароль + <code>passwordUpdatedAt</code> и отзывает все другие сессии.
<code>resetForgottenPassword</code>	То же, но без проверки старого пароля; <code>currentSessionId</code> не передаётся → отзываются все сессии пользователя.

5.8. DTO и проверка входных данных

DTO	Правила
LoginDto	email — @IsEmail; password — непустая строка
RefreshTokenDto	refreshToken — @IsJWT (должна быть похожа на JWT)
TwoFactorDto	otp — непустая строка
TwoFactorVerifyLoginOtpDto	otp, twoFactorToken — непустые строки
ForgotPasswordDto	email — @IsEmail; otp — необязательная строка (используется и для шага 1, и для шага 2)
ResetPasswordDto	resetToken — строка; password — непустая строка (без проверки длины, см. часть 9)
ChangePasswordDto	currentPassword — непустая; password — от 8 до 72 символов
UpdateUserDto	все поля необязательны: fullName ≤ 120, email (@IsEmail), phone ≤ 32, profileImage

Лимит 72 для пароля неслучаен: bcrypt учитывает только первые 72 байта пароля.

5.9. Прочее: User-Agent, почта, фильтр ошибок

- `@ParseUserAgent()` (`common/decorators/user-agent.decorator.ts`) — парсит заголовок `User-Agent` библиотекой `ua-parser-js`; результат (`os`, `browser`, `device`) сохраняется в сессию. `@Ip()` даёт IP.
- `EmailService` — отправляет письма через Resend: `sendOtp2FAuser` и `sendOtpForgotPassword`. При ошибке отправки — 500 «Failed to send OTP email».
- `AllExceptionsFilter` — единый формат ошибок; 4xx логируются как `warn`, 5xx — как `error` со стеком.

6. Фронтенд по слоям (platform-admin)

6.1. Карта файлов фронтенда

Файл	Роль
<code>src/lib/storage.ts</code>	Обёртка над <code>localStorage</code> (<code>ACCESS_TOKEN</code> , <code>REFRESH_TOKEN</code> , <code>USER_TYPE</code>) с <code>try/catch</code>
<code>src/lib/axios.ts</code>	Два <code>axios</code> -клиента, <code>interceptors</code> , автоматический <code>refresh</code>
<code>src/api/auth.ts</code>	Объект <code>authApi</code> : по функции на каждый эндпоинт
<code>src/store/auth/authSlice.ts</code>	Redux: состояние, <code>thunks</code> , редьюсеры
<code>src/store/auth/auth.types.ts</code>	Тип <code>AuthState</code>
<code>src/types/auth.ts</code> , <code>types/user.ts</code>	Типы запросов/ответов, <code>type guards</code> <code>isCompleteLoginResponse</code> , <code>isTwoFactorRequiredResponse</code>
<code>src/routes/index.tsx</code>	Таблица маршрутов (<code>/auth/*</code> публичные, <code>/dashboard/*</code> защищённые)
<code>src/components/ProtectedRoute.tsx</code>	Не пускает без токена, загружает профиль
<code>src/layouts/AuthLayout.tsx</code>	Обёртка экранов входа; если токен есть — уводит на <code>/dashboard</code>
<code>src/pages/auth/*</code>	<code>Login</code> , <code>Verify2FaOtp</code> , <code>ForgotPassword</code> , <code>VerifyForgotOtp</code> , <code>ResetPassword</code>
<code>src/pages/dashboard/account/*</code>	Профиль, смена пароля, 2FA, список сессий
<code>src/components/AppSidebar.tsx</code>	Кнопка выхода (<code>fetchLogout</code>)
<code>src/lib/redirect.ts</code>	<code>isSafeRedirectPath</code> — защита от <code>open redirect</code>
<code>src/hooks/useOtpCountdown.ts</code>	Таймер 5 минут для кода сброса пароля

6.2. Хранение токенов: `lib/storage.ts`

- Токены лежат в **localStorage** под ключами `ACCESS_TOKEN` и `REFRESH_TOKEN`; там же `USER_TYPE`.
- Каждая операция обернута в `try/catch` (`safeGetItem`, `safeSetItem`...): если хранилище недоступно (приватный режим, запрет `cookies`), приложение не падает — вход просто не переживёт перезагрузку.
- При старте `initialState` в `authSlice` читает токены из `storage` — поэтому после F5 пользователь остаётся «вошедшим».

6.3. lib/axios.ts: сердце фронтенд-авторизации

```
export const Axios = axios.create({ baseURL }); // основной клиент
const refreshClient = axios.create({ baseURL }); // отдельный – только для /auth/refresh

const PUBLIC_AUTH_PATHS = ["/auth/login", "/auth/refresh"];

Axios.interceptors.request.use((config) => {
  if (isPublicAuthPath(config.url)) return config; // у логина токена нет
  const t = storage.getAccessToken();
  if (t) config.headers.Authorization = `Bearer ${t}`;
  return config;
});

let pendingRefresh: Promise<string> | null = null;
function fetchRefreshToken() { // "single-flight"
  pendingRefresh ??= requestTokenPair().finally(() => { pendingRefresh = null; });
  return pendingRefresh;
}

Axios.interceptors.response.use(r => r, async (error) => {
  const req = error.config;
  const hadAuthHeader = Boolean(req?.headers?.Authorization);
  if (error.response?.status !== 401 || !req || req._retry || !hadAuthHeader)
    return Promise.reject(error);
  req._retry = true;
  try { token = await fetchRefreshToken(); }
  catch (e) { storage.clearStorage(); window.location.href = "/auth/login"; throw e; }
  req.headers.Authorization = `Bearer ${token}`;
  return Axios(req); // повтор исходного запроса
});
```

Приём	Зачем
Отдельный <code>refreshClient</code>	Запрос на refresh не проходит через interceptors основного клиента — иначе его собственный 401 снова запустил бы refresh (бесконечный цикл).
<code>_retry = true</code>	Один запрос повторяется не более одного раза.
<code>hadAuthHeader</code>	Реагируем на 401 только для запросов, где был токен. Неверный пароль при логине тоже даёт 401, но обновлять токен тут бессмысленно.
<code>pendingRefresh</code> (single-flight)	Если 5 запросов одновременно получили 401, на сервер уйдёт один refresh, остальные ждут тот же Promise. Это критично: refresh-токен одноразовый, второй параллельный refresh с тем же токеном провалился бы.
<code>window.location.href = '/auth/login'</code>	Если обновить не удалось — полный сброс storage и жёсткий переход на вход.

6.4. Redux: authSlice

- **Состояние** (AuthState): `user`, `accessToken`, `refreshToken`, `userType`, `error`, `twoFactorToken`, `twoFactorRequired`, `sessions` и набор флагов загрузки (`isLoginLoading`, `isMeLoading`, `isLogoutLoading`, ...).
- **Thunks**: `fetchLogin`, `fetchMe`, `fetchLogout`, `updateUser`, `changePassword`, `twoFactorGenerateOtp`, `verifyEnableDisableTwoFactor`, `verify2FaLoginOtp`, `fetchSessionsList`, `revokeSession`, `revokeOtherSessions`, `fetchForgotPassword`, `fetchForgotPasswordOtpVerify`, `resetForgottenPassword`.
- Единый паттерн: `try { await authApi.x() } catch { return rejectWithValue(getAxiosErrorMessage(...)) }`. Страница вызывает `dispatch(thunk).unwrap()` в `try/catch` и показывает текст ошибки.
- `getAxiosErrorMessage` (`lib/api.error.ts`) достаёт `message` из ответа бэкенда (строка или массив — тогда берётся первый элемент), иначе подставляет запасной текст.
- Токены сохраняются в `localStorage` **внутри thunk** (`storage.setTokens`), а в Redux попадают через редьюсер.

- `fetchMe.rejected` с `unauthorized: true` (настоящий 401) — очищает токены; сетевая ошибка или 5xx — **не** разлогинивают пользователя.

Type guards. Ответ `/auth/login` бывает двух видов: «токены» или «нужна 2FA».

`isCompleteLoginResponse` и `isTwoFactorRequiredResponse` (`types/auth.ts`) проверяют форму ответа в рантайме и заодно сужают тип в TypeScript.

6.5. Маршруты и защита страниц

```
<Route path="auth" element={<AuthLayout />}> // публичные
  login | 2fa | forgot-password | forgot-password/otp | reset-password
</Route>
<Route element={<ProtectedRoute />}> // защищённые
  <Route path="dashboard" element={<DashboardLayout />}> ... my-account ...
</Route>
```

Компонент	Логика
<code>ProtectedRoute</code>	Нет <code>accessToken</code> в Redux → <code><Navigate to="/auth/login?redirect_uri=<текущий путь>"></code> . Токен есть → <code>dispatch(fetchMe())</code> ; пока <code>user === null</code> показывает «Loading...»; затем <code><Outlet /></code> .
<code>AuthLayout</code>	Если <code>accessToken</code> уже есть, делает <code>navigate('/dashboard')</code> — залогиненному не нужен экран входа.
<code>RootRedirect</code>	Корень / всегда ведёт на <code>/auth/login</code> .
<code>isSafeRedirectPath</code>	Допускает только пути вида <code>/что-то</code> : должен начинаться с <code>/</code> , но не с <code>//</code> и не с <code>/\</code> . Так <code>?redirect_uri=//evil.com</code> не превратится в переход на чужой сайт.

6.6. Экраны авторизации и как они общаются

Экран	Что делает
<code>LoginPage</code>	Controlled-инпуты; <code>fetchLogin</code> ; полный ответ → <code>navigate(redirectTo)</code> ; 2FA → <code>navigate('/auth/2fa', { state })</code> .
<code>Verify2FaOtpPage</code>	Читает <code>twoFactorToken</code> , <code>email</code> , <code>redirectTo</code> из <code>location.state</code> ; нет токена → обратно на <code>/auth/login</code> ; 6-значный <code>InputOTP</code> ; <code>verify2FaLoginOtp</code> .
<code>ForgotPasswordPage</code>	<code>fetchForgotPassword</code> → <code>navigate('/auth/forgot-password/otp', { state: { email, createdAt } })</code> .
<code>VerifyForgotOtpPage</code>	Таймер <code>useOtpCountdown(createdAt, 5)</code> ; после истечения — кнопка «Resend Otp»; <code>fetchForgotPasswordOtpVerify</code> → <code>navigate('/auth/reset-password', { state: { email, resetToken }, replace: true })</code> .
<code>ResetPasswordPage</code>	Проверка совпадения паролей на клиенте; без <code>email/resetToken</code> в state — обратно на <code>/auth/forgot-password</code> ; <code>resetForgottenPassword</code> ; экран «Password Updated».
<code>ProfileDetailsPage</code> + компоненты	<code>EditProfile</code> , <code>ChangePassword</code> , <code>TwoFactorAuthentication</code> (диалоги), <code>SessionsList</code> .

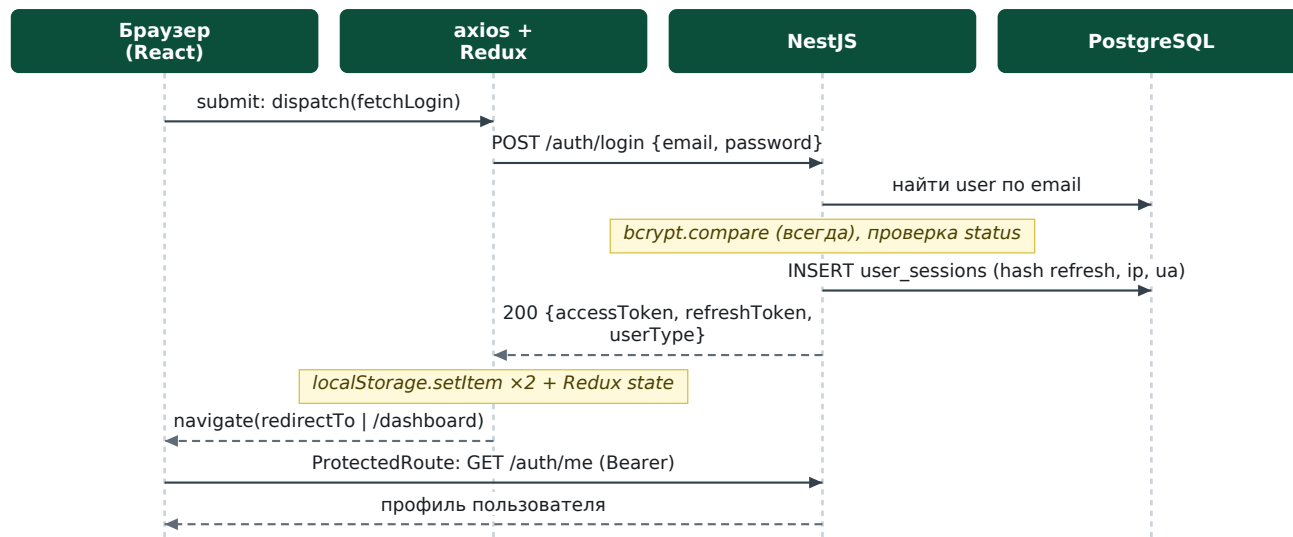
ЗАПОМНИ Промежуточные токены живут в `location.state`

`twoFactorToken` и `resetToken` передаются между экранами через состояние роутера (`navigate(..., { state })`), а не через Redux или localStorage. Они не попадают в URL (не светятся в адресной строке и истории ссылок). Зайти на `/auth/2fa` или `/auth/reset-password` напрямую по ссылке нельзя: без `state` страница сразу отправит назад.

7. Пошаговые сценарии

Это главная часть конспекта. Для каждого сценария: диаграмма, затем нумерованные шаги с указанием, **где** происходит действие.

7.1. Вход без 2FA

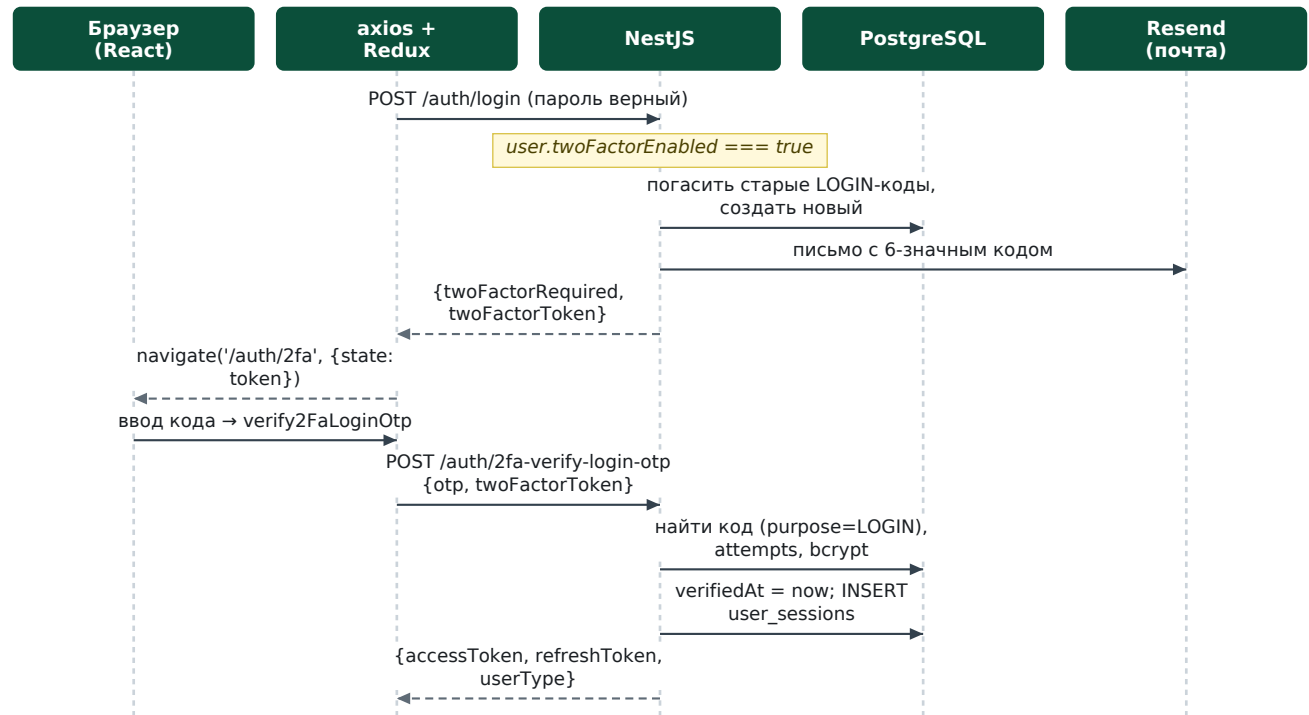


№	Где	Что происходит
1	Фронтенд	Пользователь вводит email и пароль. Поля — controlled-инпуты (<code>useState</code>). <code>type="email"</code> и <code>required</code> — только удобство UX, не безопасность.
2	Фронтенд	Submit → <code>dispatch(fetchLogin({ email, password })).unwrap() → authApi.login → POST /auth/login</code> . Interceptor видит публичный путь и не добавляет <code>Authorization</code> .
3	Бэкенд	Лимит 10 запросов в минуту; <code>ValidationPipe</code> проверяет <code>LoginDto</code> (email-формат, непустой пароль).
4	Бэкенд	<code>AuthController.login</code> собирает контекст: IP (<code>@Ip</code>) и разбор <code>User-Agent</code> (<code>@ParseUserAgent</code>) → <code>{ ipAddress, device, os, browser }</code> .
5	Бэкенд	<code>AuthService.login: findByEmail</code> ; затем <code>bcrypt.compare(password, user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH)</code> — выполняется всегда , чтобы время ответа не выдавало, существует ли email.
6	Бэкенд	Нет пользователя или пароль неверный → 401 «Invalid email or password» (одинаково в обоих случаях). Статус не <code>ACTIVE</code> → 403 «This account is not active».
7	Бэкенд	2FA выключена → <code>issueTokenPair</code> : создаётся <code>sessionId</code> , подписываются access (15 мин) и refresh (30 дней), в <code>user_sessions</code> пишется HMAC-хеш refresh-токена, <code>expiresAt</code> берётся из его <code>exp</code> .
8	Бэкенд	Ответ 200: <code>{ accessToken, refreshToken, userType }</code> .
9	Фронтенд	Thunk проверяет форму ответа (<code>isCompleteLoginResponse</code>), делает <code>storage.setTokens</code> и <code>storage.setUserType</code> . Редьюсер <code>fetchLogin.fulfilled</code> кладёт токены в Redux.
10	Фронтенд	<code>LoginPage: navigate(redirectTo, { replace: true })</code> . <code>redirectTo</code> — параметр <code>?redirect_uri</code> (если он прошёл <code>isSafeRedirectPath</code>), иначе <code>/dashboard</code> .
11	Фронтенд	<code>ProtectedRoute</code> видит токен → <code>fetchMe()</code> → <code>GET /auth/me</code> с Bearer. Пока профиль не пришёл — «Loading...», затем рендерится <code>DashboardLayout</code> .

ВНИМАНИЕ Ошибка входа

В `catch` thunk-а вызывается `storage.clearStorage()` — очищается весь `localStorage` — и `rejectWithValue(message)`. Текст ошибки показывается под формой.

7.2. Вход с двухфакторной аутентификацией



№	Где	Что происходит
1	Бэкенд	Пароль верен, но <code>twoFactorEnabled = true</code> → <code>twoFactorCreateOtpToken</code> : вызывает <code>twoFactorEnable(userId, LOGIN)</code> . Генерируется код <code>randomInt(100000, 1000000)</code> , хеш (bcrypt) пишется в <code>two_factor_otps</code> с TTL 5 минут, прежние живые коды той же цели гасятся, письмо уходит через Resend.
2	Бэкенд	Подписывается <code>twoFactorToken</code> (JWT_2FA_SECRET, 15 мин, payload { <code>userId</code> }). Токенов доступа ещё нет . Ответ: { <code>twoFactorRequired: true</code> , <code>twoFactorToken</code> , <code>message</code> }.
3	Фронтенд	<code>isTwoFactorRequiredResponse</code> → переход на <code>/auth/2fa</code> с <code>state: { twoFactorToken, email, redirectTo }</code> .
4	Фронтенд	<code>Verify2FaOtpPage</code> : пользователь вводит 6 цифр → <code>POST /auth/2fa-verify-login-otp</code> (лимит 5 в минуту).
5	Бэкенд	<code>verifyAsync(twoFactorToken)</code> : ошибка → 400 «Session is Expired». Далее: пользователь существует и ACTIVE .
6	Бэкенд	<code>verifyOtp(userId, LOGIN, otp)</code> : новейший неиспользованный и неистёкший код → иначе 400 «Otp expired or invalid»; <code>attempts ≥ 5</code> → 400 «Too many otp attempts»; неверный → <code>attempts + 1</code> и 400 «Invalid Otp»; верный → <code>verifiedAt = now</code> .
7	Бэкенд	<code>issueTokenPair</code> — дальше всё как в 7.1: создаётся сессия и возвращаются токены.
8	Фронтенд	Сохранение токенов в <code>localStorage</code> и <code>Redux</code> , <code>navigate(redirectTo)</code> .

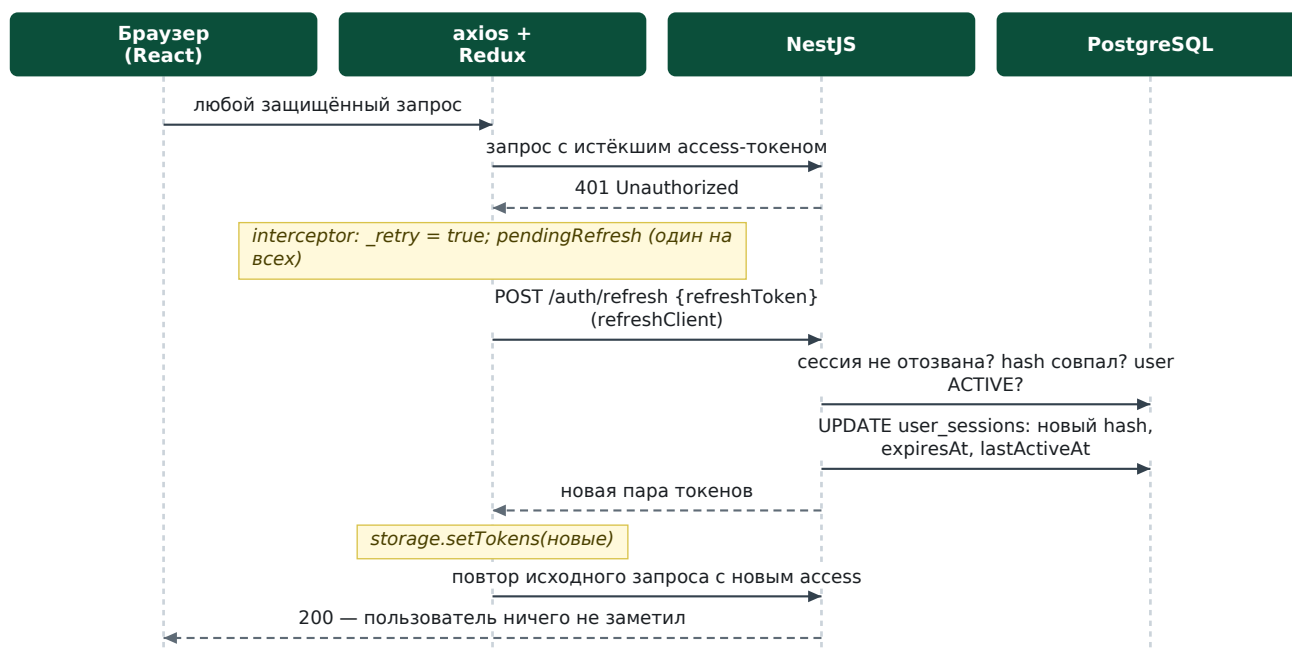
ЗАПОМНИ Зачем и `twoFactorToken`, и код

Это два разных доказательства. `twoFactorToken` (подписан сервером) доказывает, что «пароль уже проверен» — и сервер не хранит это состояние. **Код из письма** доказывает владение почтой. Без токена нельзя пробовать коды к чужой учётной записи, без кода токен бесполезен.

7.3. Запрос к защищённому эндпоинту

№	Где	Что происходит
1	Фронтенд	Request interceptor берёт <code>ACCESS_TOKEN</code> из <code>localStorage</code> и добавляет <code>Authorization: Bearer <token></code> .
2	Бэкенд	<code>ThrottlerGuard</code> (лимит) → <code>JwtAuthGuard</code> → Passport-стратегия <code>jwt</code> : достаёт Bearer-токен, проверяет подпись (<code>JWT_ACCESS_SECRET</code>) и срок (<code>ignoreExpiration: false</code>). Не прошёл → 401.
3	БД	<code>JwtStrategy.validate(payload)</code> : ищет <code>user_sessions</code> по <code>payload.sid</code> вместе с пользователем.
4	Бэкенд	Сессии нет, <code>revokedAt</code> заполнен или <code>expiresAt</code> в прошлом → 401 «Session is no longer valid». Пользователь не <code>ACTIVE</code> → 401 «This account is not active».
5	Бэкенд	В <code>request.user</code> кладётся <code>{ userId, email, userType, sessionId }; @CurrentUser()</code> отдаёт это контроллеру.
6	Бэкенд	Выполняется <code>ValidationPipe</code> , контроллер, сервис, Prisma; ответ уходит клиенту.

7.4. Автоматическое обновление токена (refresh)



№	Где	Что происходит
1	Фронтенд	Запрос получил 401. Interceptor проверяет условия: статус 401; запрос ещё не повторялся (<code>_retry</code>); в запросе был заголовок <code>Authorization</code> (т.е. это не логин).
2	Фронтенд	<code>fetchRefreshToken()</code> — если <code>refresh</code> уже идёт, все запросы ждут один и тот же Promise (<code>pendingRefresh</code>). Иначе <code>requestTokenPair()</code> берёт <code>REFRESH_TOKEN</code> и шлёт <code>POST /auth/refresh</code> через <code>refreshClient</code> .
3	Бэкенд	<code>AuthService.refresh</code> , шаг 1: <code>verifyAsync</code> по <code>JWT_REFRESH_SECRET</code> (подпись и срок) → иначе 401 «Invalid refresh token».
4	Бэкенд	Шаг 2: сессия <code>payload.sessionId</code> существует, не отозвана и не истекла.
5	Бэкенд	Шаг 3: HMAC присланного токена сравнивается с <code>refreshTokenHash</code> через <code>timingSafeEqual</code> . Не совпал → 401 (так «не работает» старый, уже использованный <code>refresh</code>).
6	Бэкенд	Пользователь должен быть <code>ACTIVE</code> (иначе 403).
7	Бэкенд	Шаг 4 — ротация : <code>signTokenPair</code> создаёт новую пару, в той же сессии обновляются <code>refreshTokenHash</code> , <code>expiresAt</code> , <code>lastActiveAt</code> , IP, ОС, браузер.

№	Где	Что происходит
8	Фронтенд	<code>storage.setTokens(новые)</code> , исходный запрос повторяется с новым access-токеном. Если refresh не удался — <code>storage.clearStorage()</code> и <code>window.location.href = '/auth/login'</code> .

ЗАПОМНИ Сессия «скользит»

Каждый успешный refresh продлевает `expiresAt` ещё на срок refresh-токена (30 дней). Если пользователь заходит регулярно, его сессия не истекает; если не появляется 30 дней — войти придётся заново.

7.5. Выход и управление сессиями

№	Где	Что происходит
1	Фронтенд	Кнопка выхода в <code>AppSidebar</code> → <code>dispatch(fetchLogout())</code> → <code>POST /auth/logout</code> (Bearer).
2	Бэкенд	<code>AuthService.logout(userId, sessionId)</code> : сессия должна принадлежать пользователю (иначе 400), затем <code>updateMany({ id, revokedAt: null }, { revokedAt: now })</code> .
3	Бэкенд	С этого момента <code>JwtStrategy</code> будет отклонять access-токен этой сессии (ему ещё могло жить до 15 минут), а refresh не пройдёт проверку сессии.
4	Фронтенд	И в <code>fulfilled</code> , и в <code>rejected</code> редьюсер очищает Redux и вызывает <code>storage.clearStorage()</code> — выход происходит локально, даже если сервер недоступен.
5	Фронтенд	Список устройств: <code>SessionsList</code> при монтировании → <code>GET /auth/sessions</code> → сервер возвращает сессии с <code>revokedAt = null</code> , новые сверху, с флагом <code>isCurrent</code> (сессия из токена).
6	Бэкенд	<code>POST /auth/sessions/:id/revoke</code> — отозвать одну сессию (проверка владельца: чужую отозвать нельзя). <code>POST /auth/sessions/revoke-others</code> — отозвать все, кроме текущей (<code>NOT: { id: currentSessionId }</code>).
7	Фронтенд	После отзыва интерфейс заново запрашивает список сессий.

7.6. Включение и выключение 2FA

№	Где	Что происходит
1	Фронтенд	<code>TwoFactorAuthentication</code> (диалог): кнопка «Send OTP» → <code>POST /auth/2fa-generate-otp</code> (Bearer, лимит 5/мин).
2	Бэкенд	<code>twoFactorEnable(userId)</code> с целью <code>ENABLE_TOGGLE</code> : код, bcrypt-хеш, TTL 5 минут, гашение прежних кодов этой цели, письмо.
3	Фронтенд	Диалог переключается на ввод 6 цифр → <code>POST /auth/2fa-verify-enable { otp }</code> (лимит 5/мин).
4	Бэкенд	<code>verifyOtp(ENABLE_TOGGLE)</code> , затем <code>twoFactorEnabled: !user.twoFactorEnabled</code> — это переключатель, а не «включить». Ответ <code>{ success, message: 'Two-factor enabled disabled', data: { twoFactorEnabled } }</code> .
5	Фронтенд	Редьюсер обновляет <code>state.user.twoFactorEnabled</code> , бейдж Enabled/Disabled меняется.

ЗАПОМНИ Зачем поле `purpose`

Код для входа (`LOGIN`) и код для переключения 2FA (`ENABLE_TOGGLE`) хранятся в одной таблице, но отличаются целью. `verifyOtp` ищет код **только своей цели** — код из письма «подтвердить вход» нельзя использовать, чтобы отключить 2FA.

7.7. Смена пароля (вы знаете текущий)

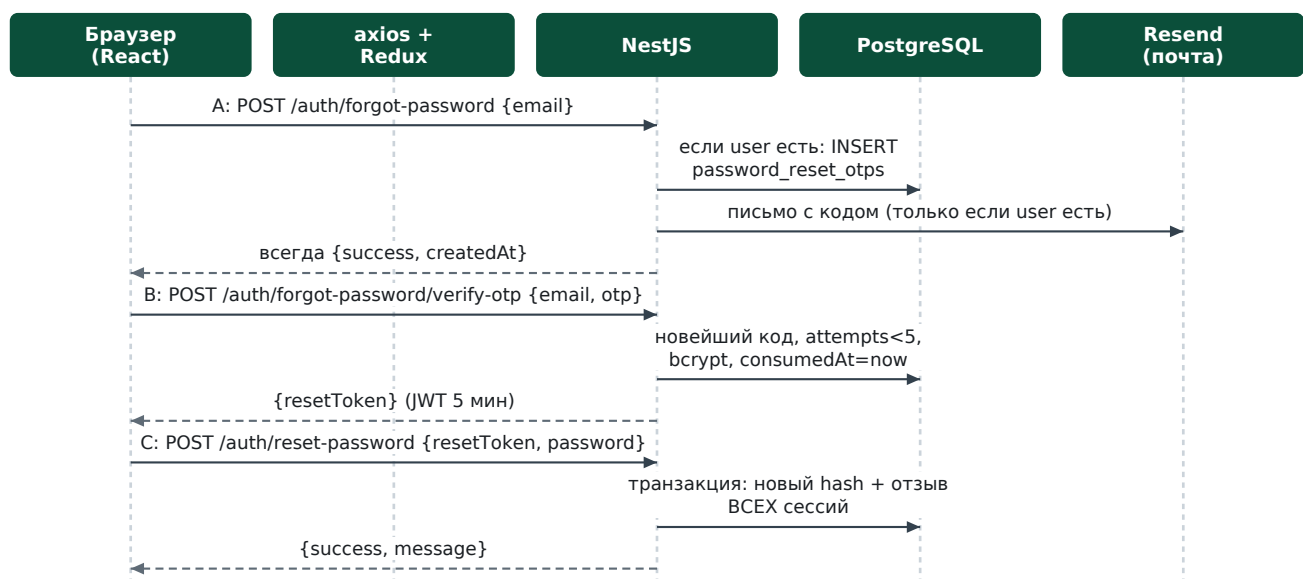
№	Где	Что происходит
1	Фронтенд	Диалог <code>ChangePassword</code> : поля текущий / новый / подтверждение. Несовпадение новых паролей ловится на клиенте. Кнопка неактивна, пока что-то не заполнено.
2	Фронтенд	<code>PUT /users/change-password { currentPassword, password } (Bearer)</code> .
3	Бэкенд	<code>ChangePasswordDto</code> : новый пароль 8–72 символа.
4	Бэкенд	<code>UserService.changePassword: bcrypt.compare(currentPassword, hash) → неверный → 401 «Current password is incorrect».</code>
5	БД	В одной транзакции : новый <code>passwordHash</code> + <code>passwordUpdatedAt</code> , и <code>revokedAt = now</code> для всех других сессий пользователя (кроме текущей, <code>id: { not: currentSessionId }</code>).
6	Фронтенд	Ответ обновляет <code>state.user</code> ; на странице аккаунта меняется подпись «changed ... ago».

ЗАПОМНИ Смена пароля выкидывает остальные устройства

Если пароль украли и злоумышленник уже вошёл, смена пароля немедленно отзовёт его сессию: его access-токен перестанет проходить `JwtStrategy`, а refresh — проверку сессии.

7.8. Восстановление пароля (забыли пароль)

Три шага, три «доказательства»: **код из письма** (владение почтой) → `resetToken` (код пройден) → **новый пароль**.



№	Где	Что происходит
1	Фронтенд	<code>ForgotPasswordPage</code> : ввод email → <code>fetchForgotPassword</code> .
2	Бэкенд	<code>forgotPassword(email)</code> : если пользователь существует — 6-значный код, bcrypt-хеш, TTL 5 минут, письмо. Ответ всегда <code>{ success: true, message, data: { createdAt } }</code> (для несуществующего email <code>createdAt</code> — просто «сейчас»). Так нельзя узнать, какие email зарегистрированы.
3	Фронтенд	Переход на <code>/auth/forgot-password/otp</code> с <code>state: { email, createdAt }</code> . Хук <code>useOtpCountdown</code> считает оставшиеся 5 минут от <code>createdAt</code> ; по истечении показывается «Resend Otp».
4	Бэкенд	<code>forgotPasswordOtpVerification(email, otp)</code> : пользователь (иначе 400 «Otp does not match»); новейший неиспользованный и неистёкший код; <code>attempts ≥ 5 → 400</code> ; неверный → <code>attempts + 1</code> ; верный → <code>consumedAt = now</code> (код одноразовый).

№	Где	Что происходит
5	Бэкенд	Подписывается <code>resetToken</code> (<code>JWT_RESET_SECRET</code> , 5 минут, <code>{ userId }</code>) и возвращается клиенту.
6	Фронтенд	Переход на <code>/auth/reset-password</code> (<code>replace: true</code>) со <code>state: { email, resetToken }</code> .
7	Фронтенд	<code>ResetPasswordPage</code> : проверка совпадения двух паролей → <code>POST /auth/reset-password { resetToken, password }</code> .
8	Бэкенд	<code>resetPassword</code> : проверка <code>resetToken</code> , затем <code>UserService.resetForgottenPassword</code> : хеш нового пароля и в транзакции отзыв всех сессий (текущей сессии у забывшего нет).
9	Фронтенд	Экран «Password Updated» и кнопка «Back to sign in». Токены не выдаются — после сброса нужно войти заново.

7.9. Редактирование профиля

№	Где	Что происходит
1	Фронтенд	<code>EditProfile</code> (диалог): форма с <code>fullName, email, phone, profileImage</code> ; кнопка активна, только если что-то изменилось.
2	Фронтенд	<code>PUT /users/me</code> (Bearer) с этими полями.
3	Бэкенд	<code>UpdateUserDto</code> + <code>whitelist</code> : лишние поля вроде <code>userType</code> или <code>status</code> отклоняются.
4	Бэкенд	<code>UserService.update</code> → Prisma <code>update</code> с <code>omit</code> . Занятый email → Prisma <code>P2002</code> → 409 «Email is already in use».
5	Фронтенд	Редьюсер <code>updateUser.fulfilled</code> обновляет <code>state.user</code> и пересчитывает инициалы аватара (<code>avatarName</code>).

8. Что в вашей реализации сделано хорошо

Это то, что стоит понимать и уметь объяснить — многие учебные проекты так не делают.

Приём	Где	Чем полезен
Пароли хешируются <code>bcrypt</code> (cost 10)	<code>login, UserService, seed.ts</code>	Утечка БД не раскрывает пароли
Dummy-хеш для несуществующего пользователя	<code>ABSENT_USER_PASSWORD_HASH</code>	Время ответа не выдаёт, есть ли email
Одинаковое сообщение «Invalid email or password»	<code>login</code>	Нет перечисления пользователей
Одинаковый ответ <code>forgot-password</code> для любого email	<code>forgotPassword</code>	То же самое для сброса пароля
Refresh в БД хранится как HMAC, не сам токен	<code>hashRefreshToken</code>	Утечка БД не даёт рабочих токенов
Сравнение токенов за постоянное время	<code>timingSafeEqual</code>	Защита от timing-атак
Ротация refresh на каждом обновлении	<code>refresh</code>	Украденный старый refresh перестает работать
Проверка сессии и статуса на каждый запрос	<code>JwtStrategy.validate</code>	Мгновенный logout и блокировка
Отдельные секреты для 4 видов токенов	<code>.env, signTokenPair</code>	Токен одного вида нельзя выдать за другой
ОТР: <code>randomInt</code> , <code>bcrypt</code> -хеш, TTL 5 мин, 5 попыток	<code>twoFactorEnable, verify0tp</code>	Код нельзя подобрать и нельзя переиспользовать

Приём	Где	Чем полезен
Поле <code>purpose</code> у кодов 2FA	<code>two_factor_otps</code>	Коды разных сценариев не смешиваются
Rate limiting на чувствительных маршрутах	<code>@Throttle</code>	Затрудняет перебор паролей и кодов
Транзакция при смене пароля и отзыве сессий	<code>\$transaction</code>	Пароль и сессии меняются атомарно
<code>ValidationPipe</code> (whitelist) + <code>omit</code>	<code>main.ts</code> , Prisma	Нет mass assignment, секреты не уходят клиенту
Проверка <code>.env</code> при старте, секреты $\geq$ 32 символов	<code>env.validation.ts</code>	Нельзя запустить с пустым/слабым секретом
helmet, CORS по списку origin	<code>main.ts</code>	Базовая защита заголовков и межсайтовых запросов
Single-flight refresh и <code>_retry</code>	<code>lib/axios.ts</code>	Нет гонок и бесконечных циклов
Безопасный <code>redirect_uri</code>	<code>isSafeRedirectPath</code>	Защита от open redirect
Единый формат ошибок без стека	<code>AllExceptionsFilter</code>	Клиент не видит внутренности сервера

9. На что обратить внимание: замечания к текущему коду

Это не «ошибки ученика», а нормальный список доработок любого проекта на этом этапе. Все пункты проверены по коду; в каждом — файл, суть и идея исправления. Расположены по важности.

ВНИМАНИЕ 1. Нет авторизации по роли (`userType` не проверяется)

Где: `jwt.strategy.ts`, `routes/index.tsx`, `authSlice.ts`

Что в коде: Бэкенд кладёт `userType` в `request.user`, но нигде нет `RolesGuard/@Roles(...)`. Фронтенд тоже не проверяет роль: после успешного входа любой пользователь попадает в `/dashboard`. В `authSlice` `userType` везде жёстко выставляется в `"PLATFORM_ADMIN"` независимо от ответа, а значение в `localStorage` (`USER_TYPE`) нигде не читается для решений о доступе. То есть **вход реализован, а разграничение прав — нет**: например, пользователь типа `CUSTOMER` с верным паролем сможет открыть админ-консоль. Сейчас практический риск невелик: остальные страницы дашборда — заглушки, а админских эндпоинтов на бэкенде ещё нет. Но как только они появятся, отсутствие `RolesGuard` станет дырой.

Что можно сделать: Добавить на бэкенде `@Roles('PLATFORM_ADMIN') + RolesGuard` (читает `request.user.userType`) на все админские маршруты; на фронтенде — `ProtectedRoute` с проверкой `user.userType`. Скрывать кнопки на фронтенде недостаточно: истинная защита — только на сервере.

ИДЕЯ 2. Токены лежат в `localStorage`

Где: `lib/storage.ts`

Что в коде: Любой JavaScript на странице (например, внедрённый через XSS или зависимость) может прочитать оба токена, включая 30-дневный refresh.

Что можно сделать: Для production часто хранят refresh в cookie с флагами `HttpOnly; Secure; SameSite`, а access — в памяти. Это перестройка и бэкенда (выдача cookie, `credentials: true` уже включён в CORS), и фронтенда. Для учебного проекта `localStorage` приемлем — главное понимать компромисс.

ВНИМАНИЕ 3. Сброс пароля: слабые места последнего шага

Где: `auth.service.ts` → `resetPassword, dto/reset-password.dto.ts`

Что в коде: (а) `resetToken` — JWT на 5 минут, он **не одноразовый**: до истечения им можно воспользоваться повторно. (б) `verifyAsync` не обернут в `try/catch`: при просроченном или битом токене бросается ошибка `jwt`, не являющаяся `HttpException`, и клиент получит 500 «Internal server error» вместо 400/401; проверка `if (!decode)` после него недостижима. (в) `ResetPasswordDto` проверяет только «непустая строка» — в отличие от `ChangePasswordDto` (8–72 символа); `ResetForgottenPasswordDto` с правилами 8–72 используется лишь как тип в сервисе, а не как DTO запроса в контроллере.

Что можно сделать: Обернуть `verifyAsync` в `try/catch` с `BadRequestException`; в `ResetPasswordDto` добавить `@MinLength(8)` `@MaxLength(72)`; при желании сделать токен одноразовым (хранить `jti`/признак использования в БД).

ВНИМАНИЕ 4. Страница 2FA-входа следит не за тем флагом загрузки

Где: `Verify2Fa0tpPage.tsx` и `authSlice.ts`

Что в коде: Страница читает `state.auth.isTwoFactorVerify0tpLoading` (флаг диалога включения 2FA), а запрос входа выставляет `isVerify2FaLogin0tpLoading`. В результате кнопка и поле ввода на этой странице не блокируются на время запроса.

Что можно сделать: Заменить селектор на `isVerify2FaLogin0tpLoading`. Это мелочь, но хороший пример, как важно называть флаги однозначно.

ВНИМАНИЕ 5. Параметр otp необязателен в DTO шага 2 восстановления

Где: `login-dto.ts` → `ForgotPasswordDto, auth.controller.ts`

Что в коде: Один DTO обслуживает и шаг 1 (только `email`), и шаг 2 (`email + otp`), поэтому `otp` помечен `@IsOptional`, а контроллер использует `dto.otp!`. Запрос без `otp` пройдёт валидацию и дойдёт до `bcrypt.compare(undefined, ...)`, что, вероятнее всего, закончится 500.

Что можно сделать: Сделать два DTO (`ForgotPasswordDto` и `VerifyForgotPassword0tpDto` с обязательным `otp`).

ИДЕЯ 6. Нет «обнаружения повторного использования» refresh-токена

Где: `auth.service.ts` → `refresh`

Что в коде: Ротация есть: старый refresh перестает подходить (хеш не совпадает) — запрос просто получает 401. Но сессия при этом не отзывается, так что если токен украли и использовали раньше владельца, система об этом не узнает.

Что можно сделать: Классическая схема: при попытке использовать **уже использованный** refresh — отозвать всю сессию (или семейство сессий) и считать это признаком компрометации.

ВНИМАНИЕ 7. Seed создаёт администратора с простым фиксированным паролем

Где: `backend/prisma/seed.ts`

Что в коде: Для `PLATFORM_ADMIN` в коде защиты email и очень простой пароль из 8 цифр; при повторном запуске seed он **перезаписывается** (`update: { passwordHash }`).

Что можно сделать: Брать пароль из переменной окружения, не запускать seed на production-базе без смены пароля, а у такой учётной записи обязательно включить 2FA.

ИДЕЯ 8. Мелкие замечания

Где: разные файлы

Что в коде: • `lastActiveAt` обновляется только при refresh (раз в ~15 минут), а не при каждом запросе — в списке устройств «активность» будет приблизительной. • `listSessions` не фильтрует по `expiresAt`, поэтому истёкшие, но не отозванные сессии могут попасть в список. • Смена email в `PUT /users/me` не требует ни пароля, ни подтверждения почты. • В HTML письма имя пользователя подставляется без экранирования (`Hello ${userName}`). • Пустые `create-auth.dto.ts` и `update-auth.dto.ts`, поле `twoFactorSecret`, тип `Role` на фронтенде пока не используются. • Опечатки в именах (`clearStoradge`, `ForgotPassowordPage`, `commision-payouts`) — безвредны, но их лучше исправить, пока код маленький.

Что можно сделать: Разобрать по мере роста проекта.

9.1. Что нужно для настоящей мультитенантности

- Таблица `Tenant` (магазин/организация) и связь пользователя с тенантом (в том числе многие-ко-многим, если один человек работает в нескольких магазинах).
- `tenantId` в сессии и/или в access-токене, а также в **каждой** доменной таблице (товары, заказы...).
- Серверный фильтр по `tenantId` во всех запросах (например, через Prisma extension или базовый репозиторий) — чтобы забыть `where: { tenantId }` было невозможно.
- Роли внутри тенанта (владелец, менеджер...) и `RolesGuard/PermissionsGuard`; роль платформенного админа — отдельно.
- Тесты на утечки между тенантами: пользователь тенанта А не должен видеть и менять данные тенанта В.

10. Что запомнить и как проверить себя

10.1. Чек-лист: 15 главных мыслей

1. **Аутентификация ≠ авторизация.** Вход (кто ты) у вас реализован; права (что тебе можно) — ещё нет.
2. Access-токен — короткий (15 мин), в каждом запросе. Refresh-токен — длинный (30 дней), только для обновления.
3. JWT **подписан, но не зашифрован**: payload читаем всеми, секреты в нём хранить нельзя.
4. У каждой сессии есть строка в `user_sessions`; access-токен содержит `sid` — ссылку на неё.
5. `JwtStrategy.validate` на каждом запросе проверяет сессию в БД: поэтому logout, смена пароля и блокировка работают сразу.
6. В БД лежит **HMAC-хеш** refresh-токена, а не сам токен; сравнение через `timingSafeEqual`.
7. Refresh **ротится**: после обновления старый refresh не подходит.
8. На фронтенде один refresh на все параллельные запросы (`pendingRefresh`) — иначе гонка ломает вход.
9. `refreshClient` отдельный от `Axios`, чтобы refresh не зацикливался сам на себе.
10. 2FA в два «доказательства»: `twoFactorToken` (пароль верен) + код из письма (владение почтой).
11. Коды: 6 цифр, bcrypt-хеш, 5 минут, 5 попыток, одноразовые; `purpose` разделяет сценарии.
12. `login` и `forgot-password` не раскрывают, существует ли email; `login` ещё и выравнивает время ответа.
13. Смена/сброс пароля отзывает сессии (все другие / все).
14. Фронтенд не принимает решений о безопасности: любая проверка в браузере — только удобство, настоящая защита на сервере.
15. Токены в localStorage удобны, но уязвимы к XSS; роли не проверяются; reset-токен не одноразовый — главные пункты доработки.

10.2. Вопросы для самопроверки

Попробуйте ответить вслух, потом сверьтесь с ответами.

Вопрос	Ответ
1. Почему нужно два токена, а не один долгоживущий?	Access уходит в каждом запросе и поэтому живёт мало — украденный быстро протухает. Refresh используется редко и только на <code>/auth/refresh</code> , поэтому может жить долго и при этом отзывается через сессию.

Вопрос	Ответ
2. Как access-токен можно «отозвать», если JWT сам по себе stateless?	В токене есть <code>sid</code> . На каждом запросе <code>JwtStrategy</code> читает сессию из БД; если <code>revokedAt</code> заполнен или сессия истекла — 401. Это добавляет запрос в БД, но даёт мгновенный отзыв.
3. Что хранится в <code>user_sessions.refreshTokenHash</code> и почему не сам токен?	HMAC-SHA256 от токена с секретом сервера. Если БД утечёт, из хеша нельзя получить рабочий refresh-токен.
4. Что произойдёт, если два запроса одновременно получают 401?	Благодаря <code>pendingRefresh</code> на сервер уйдёт один <code>/auth/refresh</code> , оба запроса дождутся нового access-токена и повторятся. Без этого второй refresh с уже использованным токеном провалился бы и разлогинил пользователя.
5. Зачем в <code>login</code> вызывается <code>bcrypt.compare</code> даже для несуществующего email?	Чтобы время ответа было одинаковым и нельзя было по скорости определить, зарегистрирован ли email.
6. Чем <code>twoFactorToken</code> отличается от кода из письма?	Токен — подписанное сервером «пароль проверен, ждём второй фактор»; код — доказательство доступа к почте. Нужны оба.
7. Зачем у кодов 2FA поле <code>purpose</code>?	Чтобы код для входа нельзя было использовать для включения/выключения 2FA и наоборот.
8. Почему <code>forgot-password</code> всегда отвечает «успешно»?	Чтобы нельзя было перебором узнать, какие email есть в системе.
9. Что происходит с другими устройствами при смене пароля? А при сбросе?	Смена: отзываются все сессии, кроме текущей. Сброс: отзываются все сессии.
10. Что делает <code>whitelist: true + forbidNonWhitelisted: true</code> и от чего защищает?	Отбрасывает/запрещает поля, не описанные в DTO. Защищает от mass assignment: клиент не сможет отправить <code>userType</code> или <code>status</code> в <code>PUT /users/me</code> .
11. Почему <code>isSafeRedirectPath</code> запрещает пути, начинающиеся с <code>//</code>?	<code>//evil.com</code> браузер трактует как адрес другого сайта; без проверки <code>?redirect_uri=</code> позволил бы увести пользователя после входа на чужую страницу (open redirect).
12. Назовите три главных слабых места текущей авторизации.	Нет проверки ролей (<code>userType</code>); токены в <code>localStorage</code> ; сброс пароля: reset-токен не одноразовый, нет <code>try/catch</code> и проверки длины пароля.
13. Чего не хватает проекту для мультитенантности в авторизации?	Сущности тенанта, <code>tenantId</code> в сессии/токене и в данных, серверной фильтрации по тенанту, ролей внутри тенанта и guard-ов.

Приложения

A. Как увидеть всё это своими глазами

- **DevTools → Network.** Откройте вкладку, войдите в приложение и посмотрите запросы `login`, `me`, `refresh`: в Payload — что отправил браузер, в Preview — что ответил сервер.
- **DevTools → Application → Local Storage.** Найдите `ACCESS_TOKEN`, `REFRESH_TOKEN`, `USER_TYPE`. Декодировать payload: `JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))` в консоли.
- **Проверить автообновление.** Поставьте `JWT_ACCESS_EXPIRES_IN="1m"` в `backend/.env`, войдите и через минуту сделайте любое действие: в Network увидите 401 → `/auth/refresh` → повтор запроса с 200.
- **Проверить отзыв.** Войдите в двух браузерах, отзовите вторую сессию в «Active sessions» — во втором браузере следующий запрос вернёт 401, а `refresh` не пройдёт.
- **Prisma Studio** (`npm run prisma:studio` в `backend/`): смотрите, как появляются строки в `user_sessions` и `two_factor_otp`s и как заполняются `revokedAt`, `attempts`, `verifiedAt`.

Запуск проекта

```
# backend/
cp .env.example .env          # заполнить DATABASE_URL, 4 секрета (>=32 символов), RESEND_*
npm install
npm run prisma:deploy          # применить миграции (или prisma:migrate в разработке)
npm run db:seed               # создать администратора (см. замечание 7 в части 9)
npm run start:dev             # http://localhost:4000

# platform-admin/
npm install
npm run dev                   # http://localhost:3002 (VITE_APP_API_BASE_URL, по умолчанию localhost:4000)
```

В. Карта файлов авторизации

Файл	Назначение
<code>backend/prisma/schema.prisma</code>	Модели и enum-ы
<code>backend/src/main.ts</code>	helmet, ValidationPipe, фильтр ошибок, CORS
<code>backend/src/app.module.ts</code>	Config, Throttler, Passport, модули
<code>backend/src/config/env.validation.ts</code>	Проверка <code>.env</code> при старте
<code>backend/src/auth/auth.controller.ts</code>	Эндпоинты <code>/auth/*</code>
<code>backend/src/auth/auth.service.ts</code>	Вся логика входа, токенов, сессий, 2FA, сброса пароля
<code>backend/src/auth/strategies/jwt.strategy.ts</code>	Проверка access-токена и сессии
<code>backend/src/auth/dto/*, types/*</code>	DTO и типы токенов/ответов
<code>backend/src/common/guards/jwt-auth.guard.ts</code>	Guard для защищённых маршрутов
<code>backend/src/common/decorators/*</code>	<code>@CurrentUser()</code> , <code>@ParseUserAgent()</code>
<code>backend/src/common/filters/all-exceptions.filter.ts</code>	Единый формат ошибок
<code>backend/src/users/*</code>	Профиль, смена пароля, сброс пароля
<code>backend/src/email/email.service.ts</code>	Письма с кодами (Resend)
<code>backend/prisma/seed.ts</code>	Создание администратора
<code>platform-admin/src/lib/axios.ts</code>	Interceptors и автообновление токена
<code>platform-admin/src/lib/storage.ts</code>	localStorage
<code>platform-admin/src/api/auth.ts</code>	Вызовы API
<code>platform-admin/src/store/auth/authSlice.ts</code>	Redux-логика авторизации
<code>platform-admin/src/routes/index.tsx</code> , <code>components/ProtectedRoute.tsx</code> , <code>layouts/AuthLayout.tsx</code>	Маршруты и защита страниц
<code>platform-admin/src/pages/auth/*</code>	Экраны входа, 2FA, восстановления пароля
<code>platform-admin/src/pages/dashboard/account/*</code>	Профиль, пароль, 2FA, сессии

С. Если нужно больше деталей

В репозитории уже есть ваши собственные материалы: [AUTH_FLOW.md](#) (разбор с диаграммами и примерами JSON «On the wire») и [AUTH_FLOW_RU.pdf](#) (русский разбор со скриншотами вкладки Network). Этот конспект собран заново по актуальному коду (коммит [ee4653a](#)) и сделан как учебный: словарь понятий, разбор по слоям, сценарии, список замечаний и вопросы для самопроверки.